Introduction to Intelligent Robotics - Robot Kinematics Homework

March 2026

Problem.1 Implementing Exponential Coordinates

In order to see the applications of the exponential map, we'll first need to implement a few fundamental equations in code. Fill in the provided `kin_func_skeleton.py` file to implement the following formulas using `numpy`. Test your implementation with the provided test cases by simply running `python kin_func_skeleton.py` in the command line. You will need this code to start the next problem.

(a) The skew $[\cdot]$ operator for rotation axes in 3D

- Input: 3×1 vector, $\omega = [\omega_x, \omega_y, \omega_z]^T$
- Output: 3×3 matrix

$$[\omega] = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \tag{4}$$

(b) Rotation matrix in 3D as a function of $\hat{\omega}$ and θ

- Input: a 3×1 vector, $\hat{\omega} = [\hat{\omega}_x, \hat{\omega}_y, \hat{\omega}_z]^T$ and scalar, θ
- Output: 3×3 matrix

$$R(\hat{\omega}, \theta) = e^{[\hat{\omega}]\theta} = I + [\hat{\omega}] \sin\theta + [\hat{\omega}]^2 (1 - \cos(\theta)) \tag{5}$$

(c) The 4x4 representations of the twists in 3D

- Input: 6×1 vector, $\mathcal{V} = [v^T, \omega^T]^T = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$
- Output: 4×4 matrix

$$[\mathcal{V}] = \begin{bmatrix} [\omega] & v \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -\omega_z & \omega_y & v_x \\ \omega_z & 0 & -\omega_x & v_y \\ -\omega_y & \omega_x & 0 & v_z \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{6}$$

(d) Homogeneous transformation in 3D as a function of twist $\mathcal{V}$ and joint angle θ

- Input: 6×1 vector, $\mathcal{V} = [v^T, \omega^T]^T = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$ and scalar θ
- Output: 4×4 matrix

$$T(\mathcal{V}, \theta) = e^{[\mathcal{V}]\theta} = \begin{cases} \begin{bmatrix} I & v\theta \\ 0 & 1 \end{bmatrix}, & \omega = 0 \\ \begin{bmatrix} e^{[\omega]\theta} & (I - e^{[\omega]\theta})([\omega]v) + \omega\omega^T v\,\theta \\ 0 & 1 \end{bmatrix}, & \omega \neq 0 \end{cases} \tag{7}$$

(e) Product of exponentials in 3D

- Input: n 6D vectors, $\mathcal{V}_1, \mathcal{V}_2, \ldots, \mathcal{V}_n$ and scalars, $\theta_1, \theta_2, \ldots, \theta_n$
- Output:

$$g(\mathcal{V}_1, \theta_1, \mathcal{V}_2, \theta_2, \ldots, \mathcal{V}_n, \theta_n) = e^{[\mathcal{V}_1]\theta_1} e^{[\mathcal{V}_2]\theta_2} \cdots e^{[\mathcal{V}_n]\theta_n} \tag{8}$$

Problem 2. Forward Kinematics for 6 DOF Manipulators

The 6 DOF Stanford Arm was developed in 1969 at a small school across the bay. It was one of the first robots to be controlled exclusively by a computer, and it was designed so that the kinematic calculations would be tractable with the limited computational power available at the time. It has 5 revolute joints and one prismatic joint (joint 3 in Figure 3 below). The figure shows the arm in its initial configuration. Let $l_0 = l_1 = 1$. You may also assume that in the initial configuration, q_w is a distance l_1 away from q_1. Implement the forward kinematics map of this robot by finding:

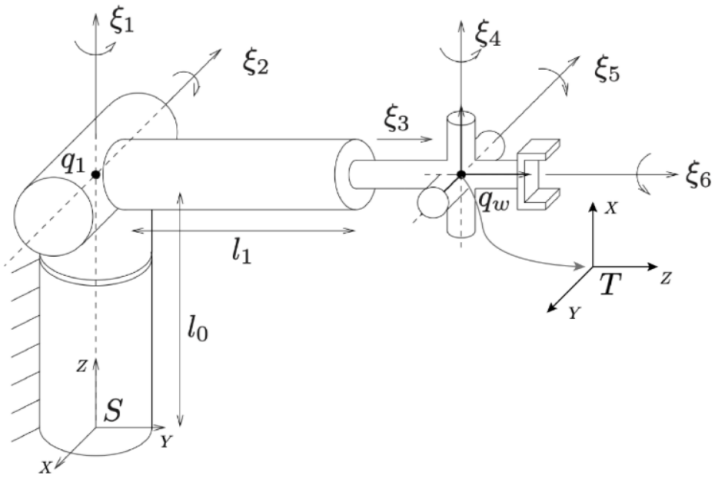

Figure 1: An idealized version of the Stanford arm.

(a) $T_{ST}(0)$, the rigid pose corresponding to the initial configuration.

(b) The twists $\xi_1, \xi_2, \xi_3, \xi_4, \xi_5, \xi_6$ (or $\mathcal{V}_i,\ i = 1, 2, \cdots, 6$ in the textbook) corresponding to each of the six joints of the manipulator.

(c) An expression for the forward kinematics map $T_{ST}(\theta)$ where $\theta \in \mathbb{R}^6$ is the vector of joint angles. You may leave your answer in terms of the exponentials of known matrices.

Problem 3. Inverse Kinematics

In practice, we often simply import Python Kinematic Toolkit to solve inverse kinematics. These toolkit will use robot description model file(i.e. ended with URDF, Unified Robot Description Format) to get each link's length and rotation axis.

In this problem, you will explore the structure of the provided ARX X5 robot model files and implement Inverse Kinematics(i.e. IK) for it in PyBullet.

The goal is to understand how a robot is described in simulation and to build a minimal control interface that allows you to manipulate the end-effector pose in real time.

(a) Understanding the model structure

Walk through the provided ARX X5 model files and answer the following:

- Identify the main XML file that defines the robot model.
- Locate the following components in the XML:
  - Bodies and kinematic tree structure
  - Joints (type, axis, limits)
  - Geometries (visual vs collision)
  - Actuators (position / velocity / torque control)
- Find the definition of the end-effector (tool frame).
- Briefly describe how the robot's kinematic chain is constructed in the XML file.

(b) Loading the robot in PyBullet

Using Python, load the ARX X5 robot model into PyBullet and visualize it.

- Initialize the PyBullet model and data using the provided XML file.
- Launch a viewer and verify that the robot is correctly rendered. Provide a screenshot of it, together with your name and student ID.

(c) Keyboard control of end-effector pose (6 DOF)

Implement an interactive control system that allows you to move the end-effector in real time using the keyboard.

- Control the following 6 DOF:
  - Translation: x, y, z
  - Rotation: roll, pitch, yaw
- Map keyboard inputs to incremental changes in desired end-effector pose.
- Convert the desired end-effector motion into joint commands through IK.
- Update the robot in real time inside the PyBullet simulation loop.

Deliverables:

- A short description of the model structure
- Python code for loading and controlling the robot
- A short video or GIF demonstrating interactive control